Laboratorium 3: Operatory, rekurencja, typ łańcuchowy w języku Java

Dominik Delimata

Informatyka

1 rok

Laboratorium nr 1

ZADANIA

1. Operatory

Zadanie 1.1 Co zwróci poniższy program. Uzasadnij odpowiedź.

```
public class IncDec{
public static void main(String[] arg){
double a=12.12;
System.out.println(a--);
System.out.println(a++);
System.out.println(--a);
System.out.println(++a);
}}
```

Program wypisze (w typowym formacie double) takie wartości, każdą w nowej linii:

1. 12.12
2. 11.12
3. 11.12
4. 12.12

Dzieje się tak, bo **postfiksowe** a--/a++ zwracają najpierw starą wartość, a dopiero potem zmieniają zmienną, a **prefiksowe** --a/++a najpierw zmieniają zmienną i dopiero zwracają nową wartość.

Zadanie 1.2 Napisz program demonstrujący działanie operatorów logicznych koniunkcji `&&` i alternatywy `||`. Użyj w tym celu konstrukcji warunkowej „if”.

```
public class Main {
    public static void main(String[] args) {
        int a = 5;
        int b = 0;

        // && : true tylko gdy oba warunki są true
        if (a > 0 && a < 10) {
            System.out.println("a jest w przedziale (0, 10)");
        } else {
            System.out.println("a nie jest w przedziale (0, 10)");
```

```java
        }

        // || : true gdy przynajmniej jeden warunek jest true
        if (a < 0 || a == 5) {
            System.out.println("a jest ujemne LUB równe 5");
        } else {
            System.out.println("a nie jest ujemne i nie jest równe 5");
        }

        if (b != 0 && (a / b) > 1) {
            System.out.println("To się nie wykona");
        } else {
            System.out.println("Bezpiecznie: nie dzielimy przez 0");
        }

        if (a == 5 || (a / b) > 1) {
            System.out.println("Pierwszy warunek true, więc drugi nie jest
liczony");
        }
    }
}
```

```
Run    Main ×
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe "-javaage
a jest w przedziale (0, 10)
a jest ujemne LUB równe 5
Bezpiecznie: nie dzielimy przez 0
Pierwszy warunek true, więc drugi nie jest liczony
```

Zadanie 1.3 Czym różnią się operatory

- • && od &
- • || od |

Napisz programy pokazujące różnice w ich działaniu.

```java
public class OperatoryLogiczne {

    static int licznik = 0;

    static boolean sprawdz(boolean wynik) {
        licznik++;
        System.out.println("  -> sprawdz() wywołane, zwraca: " + wynik);
        return wynik;
    }

    public static void main(String[] args) {

        // --- && (short-circuit AND) ---
        System.out.println("=== false && sprawdz(true) ===");
        licznik = 0;
        if (false && sprawdz(true)) { }
        System.out.println("Ile razy wywołano sprawdz(): " + licznik);
        // Wynik: 0 - prawa strona w ogóle nie jest sprawdzana!

        // --- & (bez short-circuit) ---
        System.out.println("\n=== false & sprawdz(true) ===");
        licznik = 0;
```

```java
        if (false & sprawdz(true)) { }
        System.out.println("Ile razy wywołano sprawdz(): " + licznik);
        // Wynik: 1 - prawa strona ZAWSZE jest sprawdzana

        // --- || (short-circuit OR) ---
        System.out.println("\n=== true || sprawdz(false) ===");
        licznik = 0;
        if (true || sprawdz(false)) { }
        System.out.println("Ile razy wywołano sprawdz(): " + licznik);
        // Wynik: 0 - prawa strona nie jest sprawdzana!

        // --- | (bez short-circuit) ---
        System.out.println("\n=== true | sprawdz(false) ===");
        licznik = 0;
        if (true | sprawdz(false)) { }
        System.out.println("Ile razy wywołano sprawdz(): " + licznik);
        // Wynik: 1 - prawa strona ZAWSZE jest sprawdzana
    }
}
```

```
Run    OperatoryLogiczne
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java
=== false && sprawdz(true) ===
Ile razy wywołano sprawdz(): 0

=== false & sprawdz(true) ===
  -> sprawdz() wywołane, zwraca: true
Ile razy wywołano sprawdz(): 1

=== true || sprawdz(false) ===
Ile razy wywołano sprawdz(): 0

=== true | sprawdz(false) ===
  -> sprawdz() wywołane, zwraca: false
Ile razy wywołano sprawdz(): 1

Process finished with exit code 0
```

```java
public class OperatoryBitowe {
    public static void main(String[] args) {

        int a = 5;  // binarnie: 0101
        int b = 6;  // binarnie: 0110

        System.out.println("a = " + a + "  (binarnie: " +
Integer.toBinaryString(a) + ")");
        System.out.println("b = " + b + "  (binarnie: " +
Integer.toBinaryString(b) + ")");

        int and = a & b;  // 0101 & 0110 = 0100 = 4
        int or  = a | b;  // 0101 | 0110 = 0111 = 7

        System.out.println("\na & b = " + and + "  (binarnie: " +
Integer.toBinaryString(and) + ")");
        System.out.println("a | b = " + or  + "  (binarnie: " +
Integer.toBinaryString(or)  + ")");

        // && i || nie działają na int - to błąd kompilacji:
        // if (a && b) { }  // BŁĄD!
    }
}
```

```
Run    OperatoryBitowe
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\
a = 5  (binarnie: 101)
b = 6  (binarnie: 110)

a & b = 4  (binarnie: 100)
a | b = 7  (binarnie: 111)

Process finished with exit code 0
```

Główna różnica to **krótkie spięcie** (short-circuit): && i || mogą zakończyć sprawdzanie warunku wcześniej, a & i | zawsze liczą obie strony. Poza tym & i | działają też jako operatory bitowe na liczbach całkowitych

& na liczbach porównuje każdy bit osobno: 1 AND 1 = 1, ale 1 AND 0 = 0.
&& na liczbach całkowitych jest w ogóle **niedozwolone** w Javie (błąd kompilacji).

Przy && i || licznik pozostaje 0 — prawa strona nie jest wywoływana w ogóle.
Przy & i | licznik wynosi 1 — prawa strona zawsze się wykonuje.

Zadanie 1.4 Przestudiuj następujący artykuł:
http://jlaskowski.blogspot.com/2012/01/ooperatorach-przesuniec-w-javieprawie.html

Następnie napisz programy demonstrujące działanie operatorów:

- w lewo ze znakiem <<
- w prawo ze znakiem >>
- w prawo bez znaku >>>

```java
public class OperatoryPrzesuniecia {
    public static void main(String[] args) {

        int a = 8;    // binarnie:  00000000 00000000 00000000 00001000
        int b = -8;   // binarnie:  11111111 11111111 11111111 11111000

        System.out.println("=== << (przesunięcie w lewo) ===");
        System.out.println("a = " + a + "  (bin: " +
Integer.toBinaryString(a) + ")");
        System.out.println("a << 2 = " + (a << 2));
System.out.println("a << 3 = " + (a << 3));
        System.out.println();

        System.out.println("=== >> (przesunięcie w prawo ZE znakiem) ===");
        System.out.println("a = " + a + "  (bin: " +
Integer.toBinaryString(a) + ")");
        System.out.println("a >> 1 = " + (a >> 1));
        System.out.println("a >> 2 = " + (a >> 2));
        System.out.println();
        System.out.println("b = " + b + " (bin: " +
```

```
Integer.toBinaryString(b) + ")");
        System.out.println("b >> 1 = " + (b >> 1));
        System.out.println("b >> 2 = " + (b >> 2));
        System.out.println();

        System.out.println("=== >>> (przesunięcie w prawo BEZ znaku) ===");
        System.out.println("a = " + a);
        System.out.println("a >>> 1 = " + (a >>> 1;
        System.out.println();
        System.out.println("b = " + b);
        System.out.println("b >>> 1 = " + (b >>> 1));
        System.out.println("b >>> 28 = " + (b >>> 28));
        System.out.println("(bin po >>> 1): " + Integer.toBinaryString(b
>>> 1));

        System.out.println();
        System.out.println("=== Matematyczna zależność ===");
        System.out.println("a << 1 = a * 2 = " + (a << 1));
        System.out.println("a >> 1 = a / 2 = " + (a >> 1));
    }
}
```

```
Run    OperatoryBitowe
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\
a = 5  (binarnie: 101)
b = 6  (binarnie: 110)

a & b = 4  (binarnie: 100)
a | b = 7  (binarnie: 111)

Process finished with exit code 0
```

<< to szybkie mnożenie przez 2 (każde przesunięcie o 1 to ×2).

>> to szybkie dzielenie przez 2 z zachowaniem znaku.

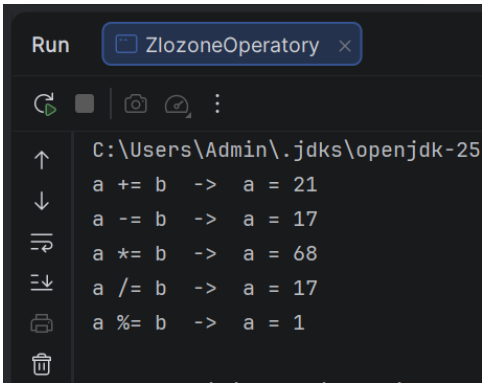
>>> używany głównie przy operacjach na bitach, gdy chcę mieć pewność, że wynik jest zawsze nieujemny (np. obliczanie środka tablicy: (left + right) >>> 1).

Zadanie 1.5 Dany jest fragment kodu:

```
int a = 17;
double b = 4.0;
a += b; // ?
a -= b; // ?
a *= b; // ?
a /= b; // ?
a %= b; // ?
```

Jakie działania arytmetyczne reprezentują wyrażenia w kodzie i jakie wyniki zwrócą?

```
public class ZlozoneOperatory {
    public static void main(String[] args) {
        int a = 17;
        double b = 4.0;

        a += b;
        System.out.println("a += b  ->  a = " + a);

        a -= b;
        System.out.println("a -= b  ->  a = " + a);

        a *= b;
        System.out.println("a *= b  ->  a = " + a);

        a /= b;
        System.out.println("a /= b  ->  a = " + a);

        a %= b;
        System.out.println("a %= b  ->  a = " + a);
    }
}
```



Ważna cecha tych wyrażeń: a jest typu int, b jest double. Operatory złożone (+=, -= itd.) wykonują **automatyczne rzutowanie** (narrowing cast) wyniku z powrotem do int, obcinając część ułamkową. Odpowiada to a = (int)(a op

Zadanie 1.6 Dany jest fragment kodu:

```
int a = 5;
int b = 3;
int c = a+b++;
int d =(a++)+b;
```

W jakiej kolejności będą wykonywana działania podczas wyliczania wartości zmiennych c i d? Jakie wartości zostaną wyliczone?

```java
public class InkrementacjaKolejnosc {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;

        int c = a + b++;
        System.out.println("c = " + c);
        System.out.println("b po c = " + b);

        int d = (a++) + b;
        System.out.println("d = " + d);
        System.out.println("a po d = " + a);
    }
}
```

```
Run    InkrementacjaKolejnosc
C:\Users\Admin\.jdks\openjdk-25.0.2\
c = 8
b po c = 4
d = 9
a po d = 6

Process finished with exit code 0
```

Oba wyrażenia korzystają z **postfiksowego** ++, który zwraca najpierw starą wartość, a dopiero **po** użyciu w wyrażeniu zwiększa zmienną o 1.

Zadanie 1.7 Jakimi operatorami w Javie definiuje się operacje logiczne NOT i XOR. Utwórz odpowiednie programy demonstrujące działanie dla tych bramek logicznych.

Bramka NOT

```java
public class BramkaNOT {
    public static void main(String[] args) {
        System.out.println("=== Bramka NOT (!) ===");
        System.out.println("Tablica prawdy:");
        System.out.println("A      | !A");
        System.out.println("------|-----");

        boolean[] wartosci = {false, true};

        for (boolean a : wartosci) {
            System.out.printf("%-5s | %-5s%n", a, !a);
        }

        System.out.println("\nPrzykłady użycia:");
        boolean deszczPada = true;

        if (!deszczPada) {
            System.out.println("Nie pada, możesz wyjść bez parasola");
        } else {
            System.out.println("Pada deszcz, weź parasol");
        }
    }
}
```

```
Run    BramkaNOT
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\
=== Bramka NOT (!) ===
Tablica prawdy:
A      | !A
------|-----
false | true
true  | false

Przykłady użycia:
Pada deszcz, weź parasol

Process finished with exit code 0
```

Bramka XOR

```java
public class BramkaXOR {
    public static void main(String[] args) {
        System.out.println("=== Bramka XOR (^) ===");
        System.out.println("Tablica prawdy:");
        System.out.println("A      | B      | A ^ B");
        System.out.println("------|-------|------");

        boolean[] wartosci = {false, true};

        for (boolean a : wartosci) {
```

```java
            for (boolean b : wartosci) {
                System.out.printf("%-5s | %-5s | %-5s%n", a, b, a ^ b);
            }
        }

        System.out.println("\nPrzykłady użycia:");

        // Przykład 1: Drzwi z dwoma przełącznikami
        boolean przelacznik1 = true;
        boolean przelacznik2 = false;
        boolean swiatloWlaczone = przelacznik1 ^ przelacznik2;

        System.out.println("\nPrzełącznik 1: " + przelacznik1);
        System.out.println("Przełącznik 2: " + przelacznik2);
        System.out.println("Światło włączone: " + swiatloWlaczone);
    }
}
```

```
Run    BramkaXOR

C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe "-
=== Bramka XOR (^) ===
Tablica prawdy:
A     | B     | A ^ B
------|-------|------
false | false | false
false | true  | true
true  | false | true
true  | true  | false

Przykłady użycia:

Przełącznik 1: true
Przełącznik 2: false
Światło włączone: true

Process finished with exit code 0
```

NOT (negacja logiczna) to operator !

XOR (alternatywa wykluczająca) to operator ^

Zadanie 1.8 Utwórz program wykorzystujący działanie operatora przeciwnego do operatora porównania ==.

```
public class OperatorRozny {
    public static void main(String[] args) {

        // Liczby
        int a = 5;
        int b = 10;
        int c = 5;

        System.out.println("=== Porównanie liczb ===");
        System.out.println("a = " + a + ", b = " + b + ", c = " + c);
        System.out.println("a != b : " + (a != b));
        System.out.println("a != c : " + (a != c));
        System.out.println("a == c : " + (a == c));

        // Znaki
        char litera1 = 'A';
        char litera2 = 'B';

        System.out.println("\n=== Porównanie znaków ===");
        System.out.println("litera1 = " + litera1 + ", litera2 = " +
litera2);
        System.out.println("litera1 != litera2 : " + (litera1 != litera2));

        // Boolean
        boolean x = true;
        boolean y = false;

        System.out.println("\n=== Porównanie boolean ===");
        System.out.println("x = " + x + ", y = " + y);
        System.out.println("x != y : " + (x != y));

        // Użycie w if
        System.out.println("\n=== Użycie w instrukcji if ===");
        int wynik = 100;
        int oczekiwany = 99;

        if (wynik != oczekiwany) {
            System.out.println("Wynik (" + wynik + ") różni się od
oczekiwanego (" + oczekiwany + ")");
        } else {
            System.out.println("Wynik jest zgodny z oczekiwanym");
        }

        // Pętla działająca dopóki a != 0
        System.out.println("\n=== != w pętli while ===");
        int licznik = 5;
        while (licznik != 0) {
            System.out.print(licznik + " ");
            licznik--;
        }
        System.out.println("\nKoniec odliczania!");
    }
}
```

```
Run    OperatorRozny
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.
=== Porównanie liczb ===
a = 5, b = 10, c = 5
a != b : true
a != c : false
a == c : true

=== Porównanie znaków ===
litera1 = A, litera2 = B
litera1 != litera2 : true

=== Porównanie boolean ===
x = true, y = false
x != y : true

=== Użycie w instrukcji if ===
Wynik (100) różni się od oczekiwanego (99)

=== != w pętli while ===
5 4 3 2 1
Koniec odliczania!
```

Relacja między == i !=

Wyrażenie	Znaczenie	Wynik gdy a=5, b=5	Wynik gdy a=5, b=10
a == b	a **równe** b	true	false
a != b	a **różne od** b	false	True

!= zawsze zwraca wartość **odwrotną** do == — to dokładnie tak jak NOT(==), czyli !(a == b).

2. Tytuł łańcuchowy

Zadanie 2.1 Zapoznaj się z dokumkentacją klasy String:

http://docs.oracle.com/javase/6/docs/api/java/lang/String.html

Przetestuj działanie poniższego programu

```
public class StringExample
{
public static void main( String args[] )
{
String s1 = new String( "Happy " );
String s2 = new String( "Birthday" );
System.out.printf( "s1 = %s\ns2 = %s\n\n",s1, s2 );
System.out.printf(
"Result of s1.concat( s2 ) = %s\n", s1.concat( s2 ) );
System.out.printf( "s1 after concatenation = %s\n", s1 );
} // end main
}
```

Tworzy dwa obiekty String. new String(...) to jawne tworzenie obiektu. Wypisuje wartości s1 i s2. %s to miejsce na tekst (String), \n to nowa linia. Wywołuje concat() — łączy s1 i s2 i **zwraca nowy String** „Happy Birthday". Wynik idzie wprost do printf, ale **nie jest zapisywany** do żadnej zmiennej. Wypisuje s1 — i tu widać efekt niemutowalności: s1 nadal ma wartość „Happy ", bo concat() **nie zmienia** oryginalnego stringa, tylko tworzy nowy.

Zadanie 2.2 Opisz słownie działanie następujących metod klasy String oraz utwórz proste aplikacje w języku Java pokazujące zasadę działania tych metod – jako wzór patrz program powyżej. Wyniki działania wypisz na konsoli.

char[] toCharArray() — zamienia String na tablicę znaków (char[]). Każda litera stringa staje się osobnym elementem tablicy.

byte[] getBytes() — zamienia String na tablicę bajtów (byte[]). Każdy znak jest zakodowany jako liczba (kod ASCII/UTF-8). Przydatne przy operacjach sieciowych i plikowych.

boolean equals(String str) — porównuje dwa Stringi **z uwzględnieniem wielkości liter**. Zwraca true jeśli są identyczne, false jeśli różne. Nie używaj == do porównywania Stringów!

boolean equalsIgnoreCase(String str) — jak equals(), ale **ignoruje wielkość liter** — „Java" i „java" są równe.

int compareTo(String str) — porównuje Stringi leksykograficznie (jak w słowniku). Zwraca 0 gdy równe, wartość ujemną gdy this < str, wartość dodatnią gdy this > str.

int compareToIgnoreCase(String str) — jak compareTo(), ale **ignoruje wielkość liter**.

int indexOf(int i) — szuka **pierwszego** wystąpienia znaku (podanego jako kod int lub char) w Stringu. Zwraca indeks lub -1 jeśli nie znaleziono.

int indexOf(String str) — szuka **pierwszego** wystąpienia podanego podciągu (Stringa). Zwraca indeks jego początku lub -1.

int lastIndexOf(int i) — szuka **ostatniego** wystąpienia znaku. Zwraca indeks lub -1.

int lastIndexOf(String str) — szuka **ostatniego** wystąpienia podciągu. Zwraca indeks lub -1.

String substring(int startIndex) — zwraca nowy String od indeksu startIndex do końca.

String substring(int startIndex, int stopIndex) — zwraca nowy String od startIndex do stopIndex (**wyłącznie** — znak na stopIndex nie jest zawarty).

String replace(char original, char zamiennik) — zamienia **wszystkie** wystąpienia znaku original na zamiennik. Zwraca nowy String.

String trim() — usuwa białe znaki (spacje, tabulatory) z **początku i końca** Stringa. Nie rusza spacji wewnątrz.

String toLowerCase() — zamienia wszystkie litery na **małe**.

String toUpperCase() — zamienia wszystkie litery na **wielkie**.

String[] split(String regex, int limit) — dzieli String według wzorca regex, maksymalnie na limit części.

String[] split(String regex) — dzieli String według wzorca regex na tyle części, ile możliwe.

3. Rekursja (rekurencja)

Zadanie 3.1

```
import java.util.*;
public class Silnia {
// metoda silnia zwraca silnię z liczby przekazanej jako parametr
// obliczenie silni odbywa się za pomocą rekurencji
public static int silnia(int wartosc) {
// jeśli przekazany parametr jest równy zero to zwróć 1
// a w przeciwnym wypadku zwróć wartość parametru * wywołanie metody silnia
// z parametrem o jeden mniejszym
if(wartosc == 0) return 1;
else return wartosc * silnia(wartosc - 1);
}
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
System.out.print("Podaj liczbę: " );
// pobieramy od użytkownika liczbę
int liczba = sc.nextInt();
// wyświetlamy na ekranie obliczoną silnię
System.out.println(liczba + "! = " + silnia(liczba));
}
}
```

```
import java.util.*;
public class Silnia {
    // metoda silnia zwraca silnię z liczby przekazanej jako parametr
// obliczenie silni odbywa się za pomocą rekurencji
    public static int silnia(int wartosc) {
// jeśli przekazany parametr jest równy zero to zwróć 1
// a w przeciwnym wypadku zwróć wartość parametru * wywołanie metody silnia
// z parametrem o jeden mniejszym
        if(wartosc == 0) return 1;
        else return wartosc * silnia(wartosc - 1);
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("Podaj liczbę: " );
// pobieramy od użytkownika liczbę
        int liczba = sc.nextInt();
// wyświetlamy na ekranie obliczoną silnię
        System.out.println(liczba + "! = " + silnia(liczba));
```

```
}
Run  Silnia
C:\Users\Admin\.jdks\openjdk-25.0
Podaj liczbę: 10
10! = 3628800

Process finished with exit code 0
}
```

Zadanie 3.2 Utwórz program obliczający silnie bez użycia rekurencji. Do wykonania działania użyj dowolnej pętli.

```java
import java.util.Scanner;

public class SilniaPetla {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Podaj liczbę: ");
        int liczba = sc.nextInt();

        if (liczba < 0) {
            System.out.println("Silnia nie istnieje dla liczb ujemnych!");
            return;
        }

        long wynik = 1;

        for (int i = 1; i <= liczba; i++) {
            wynik *= i;
        }

        System.out.println(liczba + "! = " + wynik);
    }
}
```

```
Run  SilniaPetla
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\
Podaj liczbę: 5
5! = 120

Process finished with exit code 0
```

Zadanie 3.3 Zaimplementuj algorytm potęgowania w dwu wersjach: rekurencyjnej i iteracyjnej (dwie osobne metody – funkcje). W celu wykonania zadania zapoznaj się z następującym materiałem:
http://pl.wikipedia.org/wiki/Algorytm_szybkiego_pot%C4%99gowania

```
import java.util.Scanner;

public class SzybkiePotegowanie {

    // ==============================================
    // WERSJA REKURENCYJNA
    // ==============================================
    public static long potegowanieRekurencyjne(long podstawa, int
wykladnik) {
        // warunek stopu: x^0 = 1
        if (wykladnik == 0) return 1;

        if (wykladnik % 2 == 0) {
            long polowa = potegowanieRekurencyjne(podstawa, wykladnik / 2);
            return polowa * polowa;
        }

        long polowa = potegowanieRekurencyjne(podstawa, (wykladnik - 1) /
2);
        return podstawa * polowa * polowa;
    }

    // ==============================================
    // WERSJA ITERACYJNA
    // ==============================================
    public static long potegowanieIteracyjne(long podstawa, int wykladnik)
{
        long wynik = 1;
        long baza = podstawa;
        int n = wykladnik;

        while (n > 0) {
            if (n % 2 == 1) {
                wynik *= baza;
            }

            baza *= baza;
            n /= 2;
        }
        return wynik;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Podaj podstawę: ");
        long podstawa = sc.nextLong();

        System.out.print("Podaj wykładnik: ");
        int wykladnik = sc.nextInt();

        System.out.println("\nWersja rekurencyjna:");
        System.out.println(podstawa + "^" + wykladnik + " = "
                + potegowanieRekurencyjne(podstawa, wykladnik));
```

```java
        System.out.println("\nWersja iteracyjna:");
        System.out.println(podstawa + "^" + wykladnik + " = "
                + potegowanieIteracyjne(podstawa, wykladnik));

        // Weryfikacja z Math.pow()
        System.out.println("\nWeryfikacja (Math.pow): "
                + (long) Math.pow(podstawa, wykladnik));
    }
}
```

```
Run    SzybkiePotegowanie
C:\Users\Admin\.jdks\openjdk-25.0.
Podaj podstawę: 2
Podaj wykładnik: 10

Wersja rekurencyjna:
2^10 = 1024

Wersja iteracyjna:
2^10 = 1024

Weryfikacja (Math.pow): 1024
```

Zadanie 3.4 Zaimplementuj algorytm obliczający wartość ciągu Fibonacciego. W celu wykonania zadania zapoznaj się z następującym materiałem: http://pl.wikipedia.org/wiki/Ci%C4%85g_Fibonacciego

```java
import java.util.Scanner;

public class Fibonacci {

    // ==============================================
    // WERSJA REKURENCYJNA
    // Złożoność: O(2^n) — bardzo wolna dla dużych n!
    // ==============================================
    public static long fibRekurencyjny(int n) {
        // warunki stopu (base case)
        if (n == 0) return 0;
        if (n == 1) return 1;
        return fibRekurencyjny(n - 1) + fibRekurencyjny(n - 2);
    }

    // ==============================================
    // WERSJA ITERACYJNA
    // Złożoność: O(n) — szybka nawet dla dużych n
    // ==============================================
    public static long fibIteracyjny(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;

        long poprzedni = 0;
        long obecny    = 1;

        for (int i = 2; i <= n; i++) {
            long nastepny = poprzedni + obecny;
```

```java
            poprzedni = obecny;
            obecny    = nastepny;
        }
        return obecny;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Podaj n: ");
        int n = sc.nextInt();

        if (n < 0) {
            System.out.println("n musi być >= 0");
            return;
        }
        // Wyniki
        System.out.println("\nF(" + n + ") rekurencyjnie = " +
fibRekurencyjny(n));
        System.out.println("F(" + n + ") iteracyjnie   = " +
fibIteracyjny(n));

        // Wypisanie ciagu
        System.out.println("\nCiąg Fibonacciego od F(0) do F(" + n + "):");
        for (int i = 0; i <= n; i++) {
            System.out.print("F(" + i + ")=" + fibIteracyjny(i));
            if (i < n) System.out.print(", ");
        }
        System.out.println();
    }
}
```

Run Fibonacci

```
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\Intelli
Podaj n: 10

F(10) rekurencyjnie = 55
F(10) iteracyjnie   = 55

Ciąg Fibonacciego od F(0) do F(10):
F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, F(6)=8, F(7)=13, F(8)=21, F(9)=34, F(10)=55

Process finished with exit code 0
```